

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	08 - 27 - 26	DATE SUBMITTED:	08 - 28 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 5.1	Implement Ansible roles in playbooks
1. PROCEDURE OUTPUT	
Task 1: Use when command for different distributions - In the local machine, make sure you are in the local repository directory (<i>CPE232_yourname</i>). Issue the command git pull. When prompted, enter the correct passphrase or password. Describe what happened when you issued this command. Did something happen? Why? <pre>santiago@workstation:~/CPE212-Santiago\$ git pull Already up to date. santiago@workstation:~/CPE212-Santiago\$</pre> <i>It pulled the latest version of the current branch of the remote repository and compared it to my local copy. Since I already synced it to the remote repository, it showed that my current branch is updated and in sync.</i> - Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): <i>ansible-playbook --ask-become-pass install_apache.yml</i>. After executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum." <pre>PLAY RECAP ***** * centos1 : ok=1 changed=0 unreachable=0 failed=1 skipped=0 rescued=0 ignored=0 ubuntu1 : ok=3 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0 ubuntu2 : ok=3 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0</pre>	

3. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
      when: ansible_distribution == "Ubuntu"

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
      when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

```
- name: update repository index
  apt:
    update_cache: yes
    when: ansible_distribution in ["Debian", "Ubuntu"]
```

Note: This will work also if you try. Notice the changes are highlighted.

```

skipping: [centos1]
ok: [ubuntu2]
ok: [ubuntu1]

PLAY RECAP *****
*
centos1      : ok=1    changed=0    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0
ubuntu1      : ok=4    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
ubuntu2      : ok=4    changed=1    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
  
```

The playbook skipped the CentOS host three times because the plays were specifically for Ubuntu systems.

4. Edit the *install_apache.yml* file and insert the lines shown below.

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache2 package
      apt:
        name: apache2
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: update repository index
      dnf:
        update_cache: yes
      when: ansible_distribution == "CentOS"

    - name: install apache2 package
      dnf:
        name: httpd
        state: latest
      when: ansible_distribution == "CentOS"

    - name: add PHP support for apache
      dnf:
        name: php
        state: latest
      when: ansible_distribution == "CentOS"
```

Make sure to save and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```

1  ---
2  - hosts: linuxservers
3    become: true
4    tasks:
5
6      - name: Update Repository Index Ubuntu
7        apt:
8          update_cache: yes
9          when: ansible_distribution == "Ubuntu" # for Ubuntu
10
11     - name: Update Repository Index CentOS
12       dnf:
13         update_cache: yes
14         when: ansible_distribution == "CentOS" # for CentOS
15
16     - name: Install Apache2 package Ubuntu
17       apt:
18         name: apache2
19         state: latest
20         when: ansible_distribution == "Ubuntu" # for Ubuntu
21
22     - name: Install Apache2 package CentOS
23       dnf:
24         name: httpd
25         state: latest
26         when: ansible_distribution == "CentOS" # for CentOS
27
28     - name: Add PHP support for Apache Ubuntu
29       apt:
30         name: libapache2-mod-php
31         state: latest
32         when: ansible_distribution == "Ubuntu" # for Ubuntu
33
34     - name: Add PHP support for Apache CentOS
35       dnf:
36         name: php
37         state: latest
38         when: ansible_distribution == "CentOS" # for CentOS
39

```

```

skipping: [ubuntu1]
skipping: [ubuntu2]
ok: [centos1]

PLAY RECAP *****
*
centos1      : ok=4    changed=0    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0
ubuntu1      : ok=4    changed=1    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0
ubuntu2      : ok=4    changed=1    unreachable=0    failed=0
skipped=3    rescued=0    ignored=0

```

Defining when conditions per play allowed the playbook to issue the installation commands for each specific host. They skip the plays that they can't execute. The output showed that the plays for each host were successful.

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the httpd service or the Apache HTTP server in CentOS is not yet active. Thus, you need to activate it first.

5.1 To activate, go to the CentOS VM terminal and enter the following:

systemctl status httpd

The result of this command tells you that the service is inactive.

5.2 Issue the following command to start the service:

sudo systemctl start httpd

(When prompted, enter the sudo password)

sudo firewall-cmd --add-port=80/tcp

(The result should be a success)

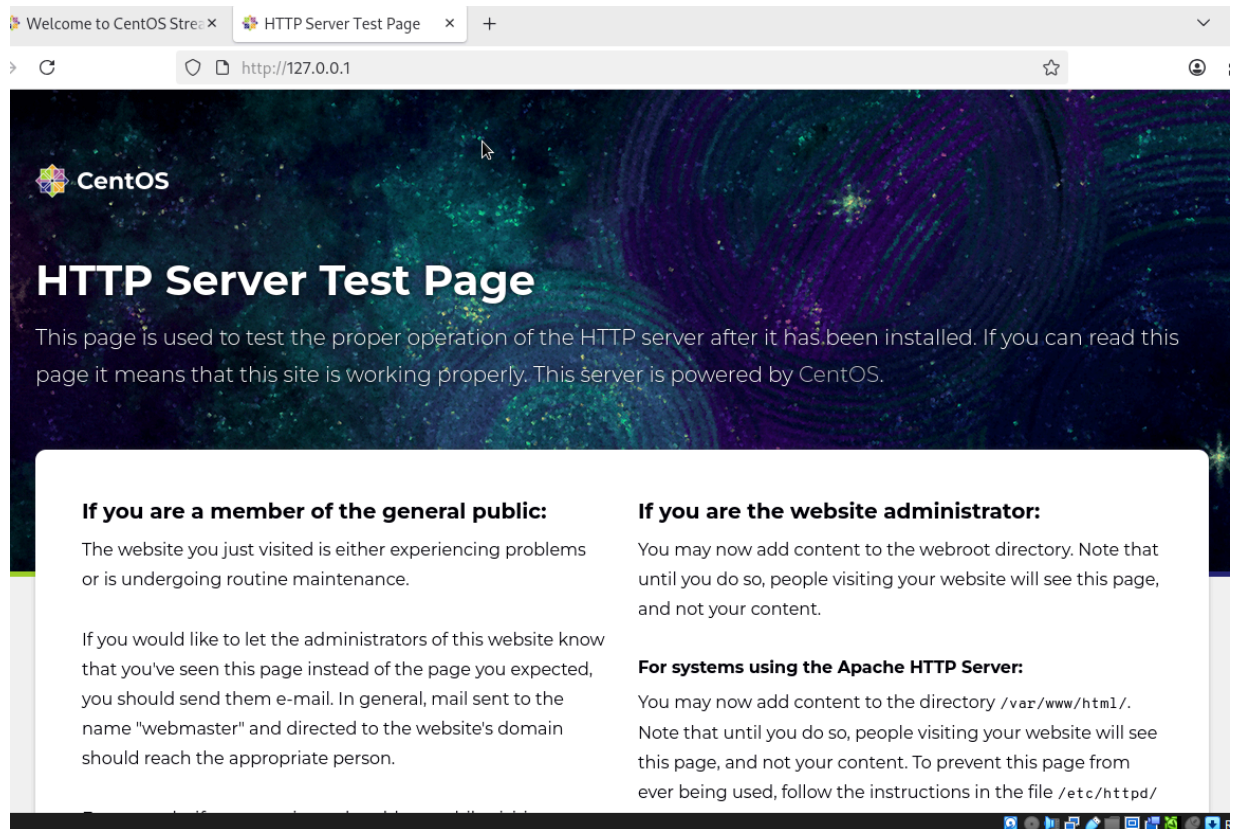
```

[santiago@centos ~]$ systemctl status httpd
o httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: d>
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: inactive (dead)
   Docs: man:httpd.service(8)
lines 1-6/6 (END)
[santiago@centos ~]$ sudo systemctl start httpd
[sudo] password for santiago:
[santiago@centos ~]$ sudo firewall-cmd --add-port=80/tcp
success

```

```
[santiago@centos ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[santiago@centos ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Thu 2026-08-27 21:10:00 EDT; 1min 30s ago
     Docs: man:httpd.service(8)
   Main PID: 7008 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0"
     Tasks: 177 (limit: 10502)
    Memory: 19.0M (peak: 19.2M)
       CPU: 110ms
    CGroup: /system.slice/httpd.service
            └─7008 /usr/sbin/httpd -DFOREGROUND
               7015 /usr/sbin/httpd -DFOREGROUND
               7020 /usr/sbin/httpd -DFOREGROUND
```

5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)



The installation and enabling of the httpd service was a success on my CentOS system.

Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also make Ansible run more quickly if it has to execute fewer tasks to do the same thing.

1. Edit the playbook *install_apache.yml*. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

```
---
- hosts: all
  become: true
  tasks:

    - name: update repository index Ubuntu
      apt:
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"

    - name: update repository index for CentOS
      dnf:
        update_cache: yes
        when: ansible_distribution == "CentOS"

    - name: install apache and php packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```

1  ---
2  - hosts: linuxservers
3    become: true
4    tasks:
5
6      - name: Update Repository Index Ubuntu
7        apt:
8          update_cache: yes
9          when: ansible_distribution == "Ubuntu" # for Ubuntu
10
11     - name: Update Repository Index CentOS
12       dnf:
13         update_cache: yes
14         when: ansible_distribution == "CentOS" # for CentOS
15
16     - name: Install Apache2 package and PHP Support Ubuntu
17       apt:
18         name:
19           - apache2
20           - libapache2-mod-php
21         state: latest
22         when: ansible_distribution == "Ubuntu" # for Ubuntu
23
24     - name: Install Apache2 package and PHP Support CentOS
25       dnf:
26         name:
27           - httpd
28           - php
29         state: latest
30         when: ansible_distribution == "CentOS" # for CentOS
31

```

```

PLAY RECAP *****
*
centos1                : ok=3    changed=0    unreachable=0    failed=0
skipped=2             rescued=0    ignored=0
ubuntu1               : ok=3    changed=1    unreachable=0    failed=0
skipped=2             rescued=0    ignored=0
ubuntu2               : ok=3    changed=1    unreachable=0    failed=0
skipped=2             rescued=0    ignored=0

```

This refactor made the code shorter and a little cleaner but did not change its core functionality. Thus, the output produced is the same as its previous version.

2. Edit the playbook *install_apache.yml* again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidate everything in just 2 plays. This can be done by removing the update repository play and putting the command *update_cache: yes* below the command *state: latest*. See below for reference:

```
---
- hosts: all
  become: true
  tasks:

    - name: install apache2 and php packages for Ubuntu
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php packages for CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
        update_cache: yes
      when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```

ansible-lab > ! install_apache.yml
1  ---
2  - hosts: linuxservers
3    become: true
4    tasks:
5
6      - name: Install Apache2 package and PHP Support Ubuntu
7        apt:
8          name:
9            - apache2
10           - libapache2-mod-php
11          state: latest
12          update_cache: yes
13          when: ansible_distribution == "Ubuntu" # for Ubuntu
14
15      - name: Install Apache2 package and PHP Support CentOS
16        dnf:
17          name:
18            - httpd
19            - php
20          state: latest
21          update_cache: yes
22          when: ansible_distribution == "CentOS" # for CentOS

```

```

skipping: [ubuntu1]
skipping: [ubuntu2]
ok: [centos1]

```

```

PLAY RECAP *****
*
centos1      : ok=2    changed=0    unreachable=0    failed=0
skipped=1   rescued=0    ignored=0
ubuntu1     : ok=2    changed=0    unreachable=0    failed=0
skipped=1   rescued=0    ignored=0
ubuntu2     : ok=2    changed=0    unreachable=0    failed=0
skipped=1   rescued=0    ignored=0

```

The output was also the same as its previous version even though the repository update was reduced to one line within the same play. It made the code shorter and easier to read but kept its core functionality.

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the `apache_package` and `php_package` are variables. The names

are arbitrary, which means we can choose different names. We also take out the line when: `ansible_distribution`. Edit the playbook *install_apache.yml* again and make sure to follow the below image. Make sure to save the file and exit.

```
---
- hosts: all
  become: true
  tasks:

    - name: install apache and php
      apt:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest
        update_cache: yes
```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
fatal: [ubuntu1]: FAILED! => {"changed": false, "msg": "Task failed: Finalization of task args for 'ansible.builtin.apt' failed: Error while resolving value for 'name': 'apache_package' is undefined"}
fatal: [ubuntu2]: FAILED! => {"changed": false, "msg": "Task failed: Finalization of task args for 'ansible.builtin.apt' failed: Error while resolving value for 'name': 'apache_package' is undefined"}
skipping: [centos1]

PLAY RECAP *****
*
centos1      : ok=1    changed=0    unreachable=0    failed=0
skipped=1   rescued=0    ignored=0
ubuntu1     : ok=1    changed=0    unreachable=0    failed=1
skipped=0   rescued=0    ignored=0
ubuntu2     : ok=1    changed=0    unreachable=0    failed=1
skipped=0   rescued=0    ignored=0
```

The output failed because the *apache_package* and *php_package* variables were not defined, thus, were unknown/empty variables.

4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the *inventory* file and follow the below configuration:

```
192.168.56.120 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.121 apache_package=apache2 php_package=libapache2-mod-php
192.168.56.122 apache_package=httpd php_package=php
```

Make sure to save the *inventory* file and exit.

Finally, we still have one more thing to change in our *install_apache.yml* file. In task 2.3, you may notice that the package is assigned as *apt*, which will not run in CentOS. Replace the *apt* with *package*. Package is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use *apt*, and for CentOS it will automatically use *dnf*. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation: [ansible.builtin.package – Generic OS package manager — Ansible Documentation](https://docs.ansible.com/ansible/latest/builtin/package_module.html)

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
1  # HOME INVENTORY FILE
2  all:
3      children:
4          ubuntu:
5              vars:
6                  apache_package: apache2
7                  php_package: libapache2-mod-php
8              hosts:
9                  ubuntu1:
10                     ansible_host: 192.168.56.102
11                     ansible_user: server1
12
13                 ubuntu2:
14                     ansible_host: 192.168.56.103
15                     ansible_user: server2
```

```

17     centos:
18         vars:
19             apache_package: httpd
20             php_package: php
21         hosts:
22             centos1:
23                 ansible_host: 192.168.56.104
24                 ansible_user: santiago
25

```

```

TASK [Install Apache and PHP] *****
*
ok: [ubuntu2]
ok: [ubuntu1]
ok: [centos1]

PLAY RECAP *****
*
centos1      : ok=2    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
ubuntu1      : ok=2    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
ubuntu2      : ok=2    changed=0    unreachable=0    failed=0
skipped=0    rescued=0    ignored=0

```

The `apache_package` and `php_package` variables are now declared in its specific OS family with the corresponding packages in the inventory. Since they are now defined, the playbook can successfully execute the installation of all packages in one play. Using `package` instead of `apt` or `dnf` also allowed this playbook to be usable to any OS as long as the OS group/hosts and corresponding package variables are defined in the inventory file.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

1. Create a playbook that could do the previous tasks in Red Hat OS.

Updated Inventory File with Redhat group to Host Red Hat OS:

```

1  # HOME INVENTORY FILE
2  all:
3    children:
4      ubuntu:
5        vars:
6          apache_package: apache2
7          php_package: libapache2-mod-php
8        hosts:
9          ubuntu1:
10           ansible_host: 192.168.56.102
11           ansible_user: server1
12
13         ubuntu2:
14           ansible_host: 192.168.56.103
15           ansible_user: server2
16
17       centos:
18         vars:
19           apache_package: httpd
20           php_package: php
21         hosts:
22           centos1:
23             ansible_host: 192.168.56.104
24             ansible_user: santiago
25
26       redhat:
27         vars:
28           apache_package: httpd
29           php_package: php
30
31     linuxservers:
32       children:
33         ubuntu:
34         centos:
35         redhat:

```

Since CentOS and Redhat use the same package manager, the CentOS installation variables should also work on Red Hat systems. However, to ensure functionality and distinction, I defined a different RedHat group specifically for Red Hat systems and added it to the linuxservers metagroup. The original install_apache.yaml file should work perfectly fine.

```

ansible-lab > ! install_apache.yml
1  ---
2  - hosts: linuxservers
3    become: true
4    tasks:
5
6      - name: Install Apache and PHP for Debian Ubuntu & Red Hat/CentOS
7        package:
8          name:
9            - "{{ apache_package }}"
10           - "{{ php_package }}"
11          state: latest
12          update_cache: yes

```

OUTPUT:

```

PLAY RECAP *****
*
centos1 : ok=2  changed=0  unreachable=0  failed=0
skipped=0  rescued=0  ignored=0
ubuntu1 : ok=2  changed=0  unreachable=0  failed=0
skipped=0  rescued=0  ignored=0
ubuntu2 : ok=2  changed=0  unreachable=0  failed=0
skipped=0  rescued=0  ignored=0

```

4. CONCLUSION/ANALYSIS/LEARNINGS

Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?

Refactoring helps debugging and readability by shortening the code while keeping its core functionality intact. This helps developers read and understand code, allowing changes to be easily applied. Refactoring also prevents programs from being hardcoded which are difficult to modify. Moreover, it helps programs become scalable with complexity; code, especially advanced systems, will inevitably change and scale depending on the system needs. Thus, we have to be able to write it in a way that makes it easy to modify and change when needed.

2. When do we use the “when” command in the playbook?

We use the “when” conditional statement with plays that would only be applied to specific hosts or under specific conditions. In the activity, “when” was used for plays that are OS specific—apt installations were used when the `os_distribution` is Ubuntu while `dnf` installations were for CentOS. This prevents errors from incompatibility issues like running apt on red hat systems; it skips plays that don’t fulfill its “when” condition.

In conclusion, this activity explores the conditional statements and code refactoring in the ansible environment. The “when” statement can be used to control which plays execute on different hosts. This prevents incompatibility issues, especially when hosts have different distributions. Additionally, this activity also explored the importance of code refactoring. By refactoring our code, it makes it shorter, easier to read, and easier to integrate and scale. The original `install_apache` file was hardcoded with many plays but the new single-play version makes it applicable to pretty much any OS so long as the inventory file has the corresponding package variables defined. All in all, this activity helped me better understand code optimization and role implementation in Ansible.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.